DiscoNet: Our First Experiment

Raw Scene ID: scene_5 · Raw Frame ID: 000066 · Agents: 1 (ego), 2, 3, 4, 5
Full Frame: 152 annotated vehicles · 0 other entities · 152 total (including all 5 agents)
110 cars · 8 police cars · 12 motorcycles · 22 bicycles

PURPOSE

Check that the trained model runs, and see how its car predictions change when vehicles share information.

WHAT WE COMPARED

Own scan only: each vehicle uses its own LiDAR measurements. Sharing with other vehicles is off.

With sharing: the same model also uses information extracted from the other vehicles' LiDAR scans.

RESULT FOR EGO VEHICLE 1

11

Own scan only

cars predicted

17

With sharing

cars predicted

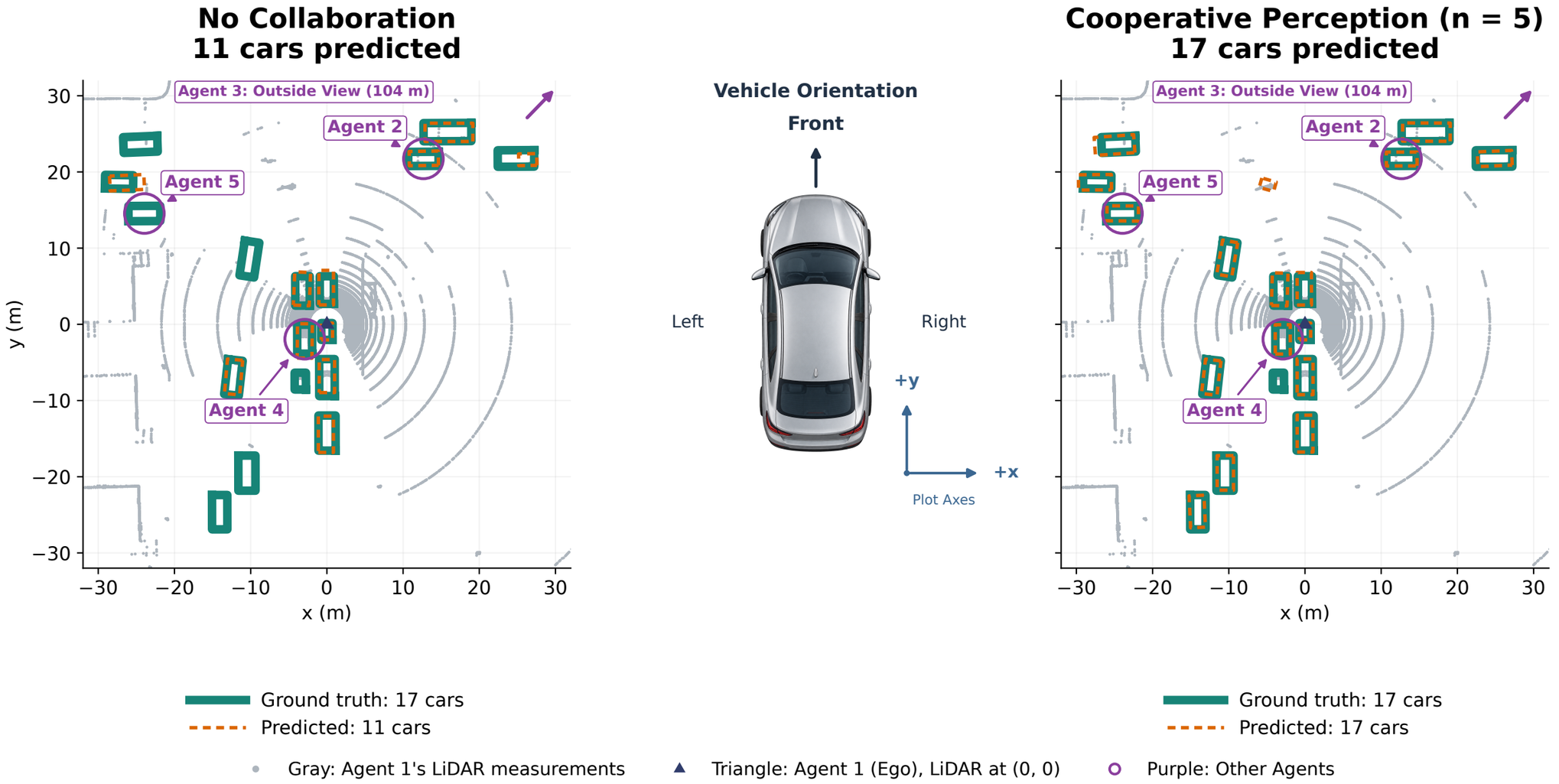
TAKEAWAY

Reference car boxes matched:
10 with own scan only
16 with cooperation

At 50% box overlap (IoU ≥ 0.5).

Example 1: Reference and Predicted Car Boxes

Plot axes: +x right, +y forward; (0, 0) is the LiDAR sensor.
Wide solid green: dataset reference (ground truth). Thin dashed orange: model predictions.



The dataset labels can include cars hidden from this vehicle's sensor.

AI generated illustration

Example 1: Camera Views

These six camera views were recorded at the same moment as the LiDAR scan in our experiment.

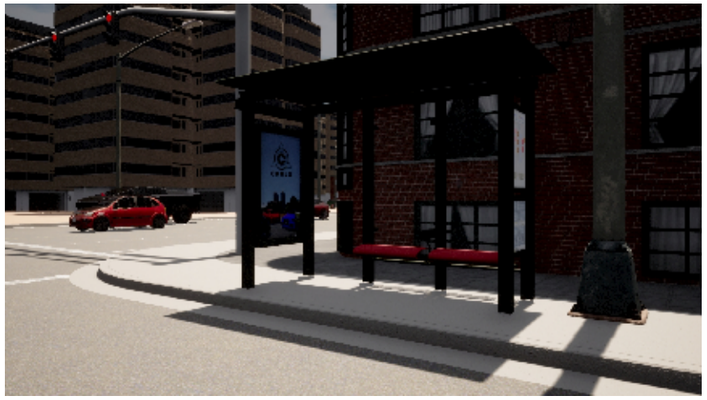
Front Left



Front



Front Right



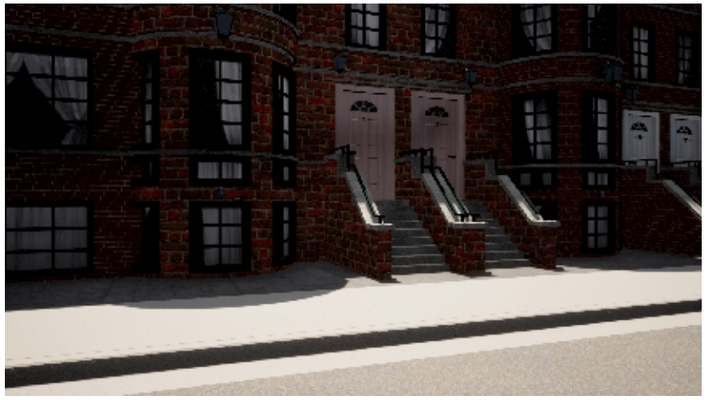
Rear Left



Rear



Rear Right



The images provide visual context; this detector used LiDAR measurements. Camera and LiDAR positions differ.

Original simulated RGB images · 1600 × 900 pixels each · V2X-Sim 2.0 · scene_5, frame 60

V2X-Sim 2.0: Dataset Overview

Recorded virtual traffic for studying how vehicles perceive the same scene together

Simulated Traffic

Created with CARLA and SUMO.
The saved recordings can be used without running the simulators.

Synchronized Vehicle Views

Vehicles and a roadside unit observe the same moment together.
Each agent has a different viewpoint.

Sensors and Reference Labels

LiDAR, camera images and vehicle poses.
Labels identify objects in the scene.
Supports detection, tracking and segmentation.

From Recorded Point Clouds to Our Model Input

Original Dataset

LiDAR point clouds and camera images, with vehicle poses and reference labels.

Preprocess →

Prepared Detection Release

LiDAR converted into bird's-eye-view grids.
The grids record which 3D cells are occupied.
We used the authors' prepared detection files.

Official scene split: 80 Train · 10 Validation · 10 Test

Our benchmark starts from the prepared vehicle LiDAR grids.

DiscoNet's encoder still computes the learned features from these grids.

Camera images provide visual context; the roadside unit is excluded from our model inputs.

Sources: V2X-Sim dataset website CoPerception data preparation

AI generated illustration

Authors' Training Setup and Dataset Splits

Published V2X-Sim 2.0 protocol for the released DiscoNet student · Roadside unit excluded

Partition	Scenes	Synchronized Frames	Vehicle Inputs per Scene	Vehicle Scans	Raw LiDAR Points
Train	80	8,000	2–5	29,200	1.0B
Validation	10	1,000	3–5	4,000	106.8M
Test	10	1,000	3–5	4,000	100.8M
Total	**100**	**10,000**	**—**	**37,200**	**1.2B**

Train: learn weights. Validation: held-out development data. Test: final accuracy evaluation.

Released Checkpoint: Epoch 100

The authors train the DiscoNet student using a teacher model; the student is used for inference.

Our experiment loads those trained weights and evaluates the 10 test scenes; no retraining.

The next table aggregates each partition by the number of vehicle inputs.

One dataset pass; raw returns before filtering/cropping. M = million; B = billion (one decimal); exact counts are in the CSV.

Published code and splits describe the protocol; the checkpoint does not contain a scene-by-scene training history.

Sources: Authors' split lists Training code / recipe Released checkpoint

Dataset Statistics by Partition and Vehicle Inputs

V2X-Sim 2.0 · One row per (partition, vehicle inputs) · Counts summed across scenes

Partition	Vehicle Inputs	Scenes	Total Frames	Total Vehicle Scans	Total Raw LiDAR Points
Train	2	4	400	800	28.9M
Train	3	37	3,700	11,100	375.4M
Train	4	22	2,200	8,800	294.8M
Train	5	17	1,700	8,500	333.9M
Validation	3	3	300	900	22.0M
Validation	4	4	400	1,600	48.4M
Validation	5	3	300	1,500	36.4M
Test	3	3	300	900	22.7M
Test	4	4	400	1,600	41.0M
Test	5	3	300	1,500	37.1M
Total	—	**100**	**10,000**	**37,200**	**1.2B**

Authors' Epoch-100 DiscoNet Student · Our Run: Test Evaluation with Fixed Weights

Vehicle inputs = nonempty prepared vehicle records per frame; roadside-unit inputs are excluded.

Frames, vehicle scans and raw LiDAR points are totals over every scene in each row; one dataset pass.

M = million; B = billion (rounded to one decimal). Raw points precede filtering/cropping; exact counts remain in the CSV.

Partition assignments follow the authors' published splits; the checkpoint does not store its scene-by-scene training history.

Sources: Authors' split lists Training code / recipe Released checkpoint

Experiment Setup: Hardware and Software

Configuration of the completed 1,000-frame benchmark

Hardware	
GPU	NVIDIA GeForce RTX 5060 Ti
GPU Memory	16 GB
CPU	AMD Ryzen 7 5700X
CPU Cores	8 cores / 16 threads
System RAM	16 GB

Software	
OS	Omarchy (Linux)
Python	3.14.8
PyTorch	2.9.1+cu128
CUDA Runtime	12.8

Model and Execution

Official DiscoNet student · Epoch 100 · Feature sharing at layer 3 · Roadside unit excluded

Single GPU · float32 precision · One synchronized frame per batch · Seed 0 · Inference only

Dataset: V2X-Sim 2.0, official prepared detection test split · 10 scenes · 1,000 frames

Experiment Procedure, Metrics, and Running Time

Offline evaluation on the given dataset; CARLA was not running

1. Read One Synchronized Frame

Load the stored LiDAR voxel grids
from 3–5 active vehicles.
Process one frame at a time.

2. Run the Same Model Twice

No Collaboration: own scan only.
Cooperative Perception: sharing on.
Use the same trained weights.

3. Measure Each Vehicle's Output

Each active agent predicts car boxes.
Match to that agent's reference boxes
in that agent's own coordinates.

1,000 Frames → 4,000 Vehicle Views → Pooled AP Scores

One vehicle view = one active agent at one frame. Match within each view, then pool all views for scoring.

The 50,047 reference car boxes count each agent's view separately; the same car can appear in multiple views.

AP (Average Precision) summarizes precision (% of predictions correct) and recall (% of reference cars found).

IoU (Intersection over Union) = overlapping box area ÷ combined area. Report AP@0.5 and AP@0.7.

Keep confidence > 0.7; remove overlapping duplicates using Non-Maximum Suppression (NMS), IoU 0.01.

Running Time for Both Conditions Together



10 min 20 sec

Total

Whole test split; excludes dataset download, initial model loading and figure export.

AI generated illustration

Original Test Split: Loss and Detection Accuracy

Authors' epoch-100 checkpoint · 10 test scenes · 1,000 frames · 4,000 vehicle views

Condition	Classification Loss ↓	Box-Localization Loss ↓	Total Loss ↓	AP@0.5 (%) ↑	AP@0.7 (%) ↑
No Collaboration	3.0K	13.8K	16.8K	49.10	43.05
Cooperative Perception	1.5K	9.6K	11.1K	70.41	64.82

K = thousand; loss values are rounded to one decimal. Exact values remain in the source CSV.

Loss Measures Model Error; Lower Is Better

Classification loss scores car/background decisions. Box-localization loss scores errors in predicted box corners.

Total loss = classification + box-localization loss, using the authors' original test functions and normalization.

Detection Quality Is Measured by AP; Higher Is Better

AP = Average Precision. IoU = box overlap; AP@0.7 requires tighter box alignment than AP@0.5.

Both conditions use the same weights. No Collaboration uses each vehicle's own scan with information sharing off.

Authors' checkpoint log: 70.4136% / 64.8239% AP. Our cooperative AP matches to two decimals.

Additional loss-evaluation pass: 2 min 31 sec. AP and its curves come from our original full-test run.

Fixed checkpoint; no weight updates. Test loss is averaged over synchronized frames using the authors' test-code normalization.

How DiscoNet Computes Confidence Scores

Confidence and score refer to the same car-class output · Both inference modes use the same calculation

1. Predict Raw Outputs

Each logit = weighted head features + bias.
Two outputs per preset box (anchor):
one for background and one for car.

2. Convert With Softmax

Convert the logits into estimated background/car probabilities summing to 1.
Use the car probability as the score.

3. Keep and Rank Boxes

Keep scores > 0.7 and remove duplicates.
Prefer higher scores when boxes overlap.
Rank retained scores to build the AP curve.

$$\text{score} = \frac{e^{z_{\text{car}}}}{e^{z_{\text{background}}} + e^{z_{\text{car}}}}$$

z is a raw output (logit), before softmax.
The car score lies between 0 and 1.

Illustrative Example: Raw Outputs to Scores

Box	Background Logit	Car Logit	Car Score	Score > 0.7?
A	1.0	3.0	0.881 (88.1%)	Keep
B	2.0	1.0	0.269 (26.9%)	Discard
C	1.0	4.0	0.953 (95.3%)	Keep

Box A: Apply the Formula

$$\frac{e^3}{e^1 + e^3} \approx 0.881$$

88.1% car confidence

If A and C survive duplicate removal:
C (0.953) ranks before A (0.881).

Reference boxes determine correctness during evaluation. A high-confidence detection can still be a false alarm.

Training uses car/background labels to learn the head's weights; our evaluation keeps those weights fixed.

How Precision–Recall Curves and AP Are Computed

AP = Average Precision · 4,000 views per condition · One view = one vehicle in one frame

1. Rank Predicted Boxes

Pool all retained predicted boxes.
Sort by confidence, highest first.
Include one more box at each step.

2. Match Within Each View

TP (true positive) = correct car match.
FP (false positive) = unmatched/duplicate.
One match per car; IoU ≥ 0.5 or ≥ 0.7.

3. Build the Curve and Its Area

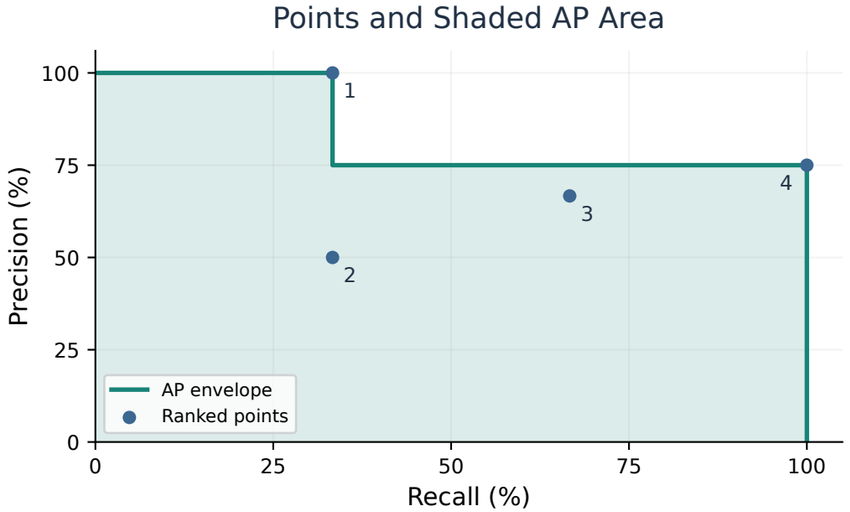
Recompute precision and recall.
Best precision at this or higher recall.
AP = area under this precision envelope.

Precision = TP / (TP + FP) **Recall = TP / Number of Reference Cars**

Illustrative Example: 3 Reference Cars

Top k	Score	Result	TP / FP So Far	Precision	Recall
1	0.99	TP	1 / 0	100.0%	33.3%
2	0.96	FP	1 / 1	50.0%	33.3%
3	0.92	TP	2 / 1	66.7%	66.7%
4	0.83	TP	3 / 1	75.0%	100.0%

Prediction 2 is a false alarm: precision falls; recall stays at 33.3%.



Example AP = (⅓ × 100%) + (⅓ × 75%) + (⅓ × 75%) = 83.3%

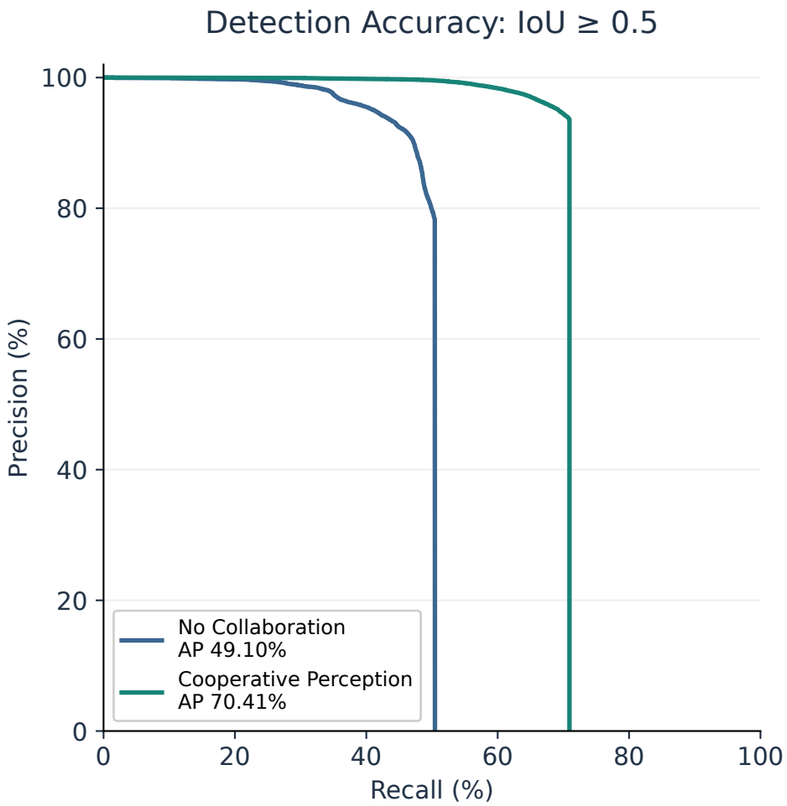
Repeat for No Collaboration and Cooperative Perception at each IoU threshold.
Real curves use the authors' score > 0.7 and duplicate-removal filters; missed cars limit recall.

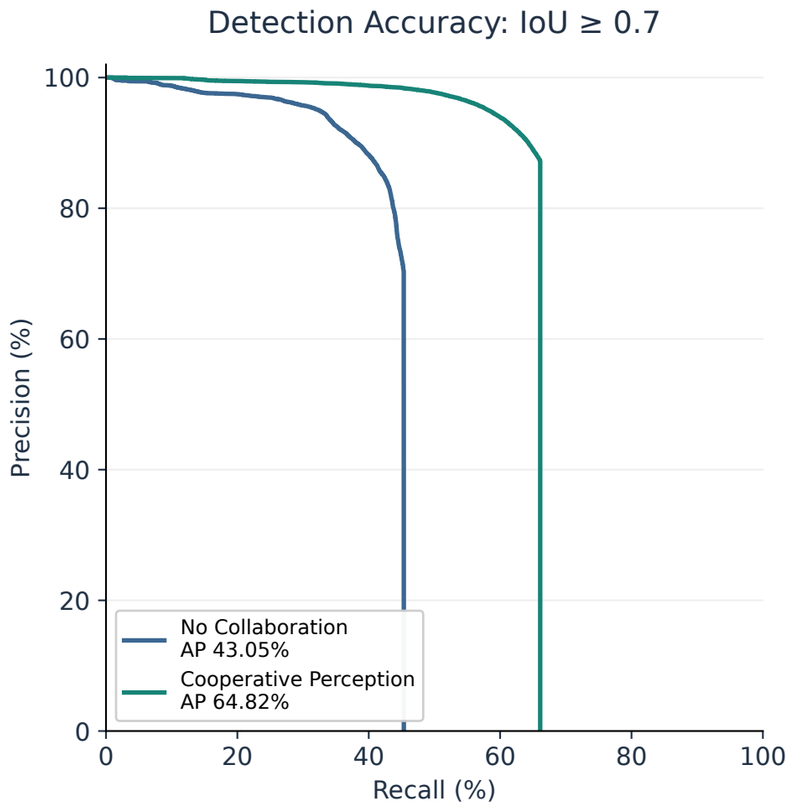
Calculation: Authors' AP code

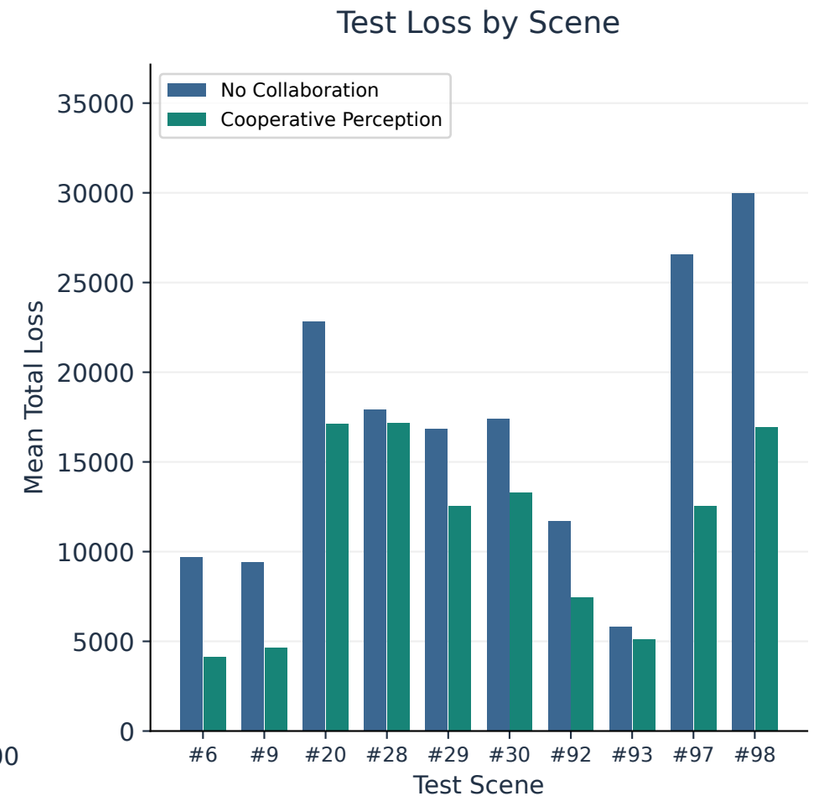
AI generated illustration

Original Test Split: Accuracy and Loss Plots

One fixed checkpoint evaluated on the authors' original test split · Information sharing off versus on







Precision: fraction of predictions correct. Recall: fraction of reference cars found. Higher accuracy curves are better.

Loss bars average 100 frames per scene. Each scene is a separate recording; lower bars indicate lower model error.

AP pools 4,000 active vehicle views. Curves use the authors' confidence cutoff > 0.7 and duplicate-removal settings.

The loss bars describe test-set evaluation of fixed weights. The checkpoint contains no epoch-by-epoch training history.

Custom Experiments

Runtime Versus Model Input Count

V2X-Sim 2.0 · n = 1–20

n = 1–5: Distinct Recorded Inputs

n = 6–20: Replayed-Input Workloads

Same trained checkpoint · One GPU · Runtime only · n counts model input slots

Custom Experiment 1: V2X-Sim Inputs and Replay

Measure runtime through n = 20 while keeping the source dataset and trained checkpoint fixed

Same Recorded Frames at Every n

Raw Scene	Frames	Recorded Vehicles
scene_9	10	5
scene_97	10	5
scene_98	10	5
Total	30	5 per frame

Vehicle IDs: 1 (ego), 2, 3, 4, 5 · No roadside inputs.

30 synchronized frames · 150 verified source files.

n Counts Model Input Slots

n = 1–5: use distinct recorded vehicle inputs.
Keep ego 1 and add the nearest recorded vehicles.
n = 6–20: cycle the five recorded inputs again.
Reuse each input's recorded pairwise poses.
Allocate n inputs and n active model slots.

Example: n = 20

Vehicle inputs [1, 2, 3, 4, 5] repeated four times.
Five recorded vehicles supply twenty model slots.

Inputs are the authors' prepared occupancy grids, derived from synchronized V2X-Sim 2.0 LiDAR recordings.

Above five, this is a runtime workload test with repeated inputs; it does not add newly recorded vehicles.

All n values are timed again on this same 30-frame cohort using unchanged epoch-100 weights.

Source: Original prepared V2X-Sim 2.0 detection test split and the verified runtime-workload record

Custom Experiment 1: Measuring Runtime

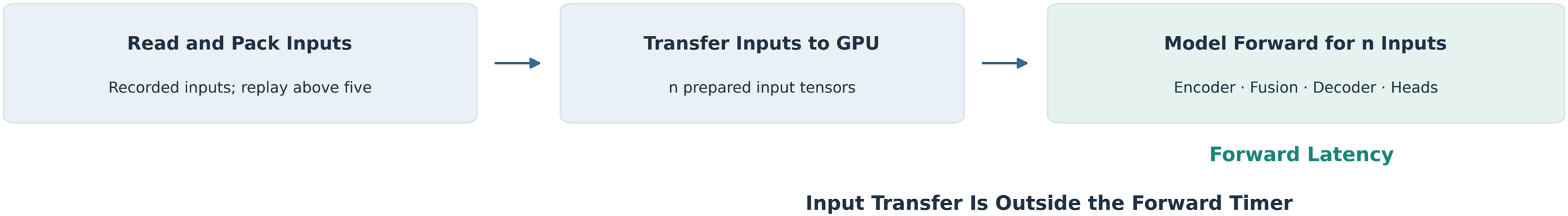
NVIDIA GeForce RTX 5060 Ti · Same checkpoint · Float32 · One full n-input group per measurement

Time the Entire Model Workload

Forward covers the encoder, pairwise feature fusion,
decoder and detection heads for all n input slots.
Start with input tensors already on the GPU.
Synchronize CUDA before and after the forward.
Report median inference time for the complete group.

Repeat Every Condition

Same 30 frames at every n from 1 through 20.
3 trials × 30 frames = 90 timing samples per n.
20 warmup forwards before each n in every trial.
Shuffle condition order between trials with seed 0.
1,800 timed forwards + 1,200 warmup forwards.



Median is the middle timing value across the 90 measured frames and trials for each n.

File reads, CPU packing, raw LiDAR preprocessing, warmups, model loading and output checks are outside the forward timer.

Runtime only: AP and box postprocessing are not computed. Wireless transmission is outside the measurement.

AI generated illustration

Custom Experiment 1: Runtime Results for n = 1–10

V2X-Sim 2.0 · Same 30 frames · Three trials · 90 timing samples per n · n counts model input slots

n Model Inputs	Recorded Vehicles	Replayed Inputs	Forward Median (ms)
1	1	0	7.8
2	2	0	13.6
3	3	0	21.8
4	4	0	31.6
5	5	0	41.4
6	5	1	52.5
7	5	2	66.3
8	5	3	80.0
9	5	4	93.2
10	5	5	103.5

Recorded Vehicles counts distinct source scans; Replayed Inputs counts additional copies of those scans.

All rows use n active model slots on one GPU. n = 6–20 is a replayed-input runtime workload.

Complete n = 1–20 sweep: 7 min 29 sec after device setup · Runtime only; no AP.

Median forward time includes encoder, collaboration, decoder and heads; input transfer, file reads and CPU packing are excluded.

Custom Experiment 1: Runtime Results for n = 11–20

V2X-Sim 2.0 · Same 30 frames · Three trials · 90 timing samples per n · n counts model input slots

n Model Inputs	Recorded Vehicles	Replayed Inputs	Forward Median (ms)
11	5	6	118.6
12	5	7	136.3
13	5	8	150.4
14	5	9	169.7
15	5	10	186.6
16	5	11	206.6
17	5	12	238.9
18	5	13	260.4
19	5	14	283.0
20	5	15	306.0

Recorded Vehicles counts distinct source scans; Replayed Inputs counts additional copies of those scans.

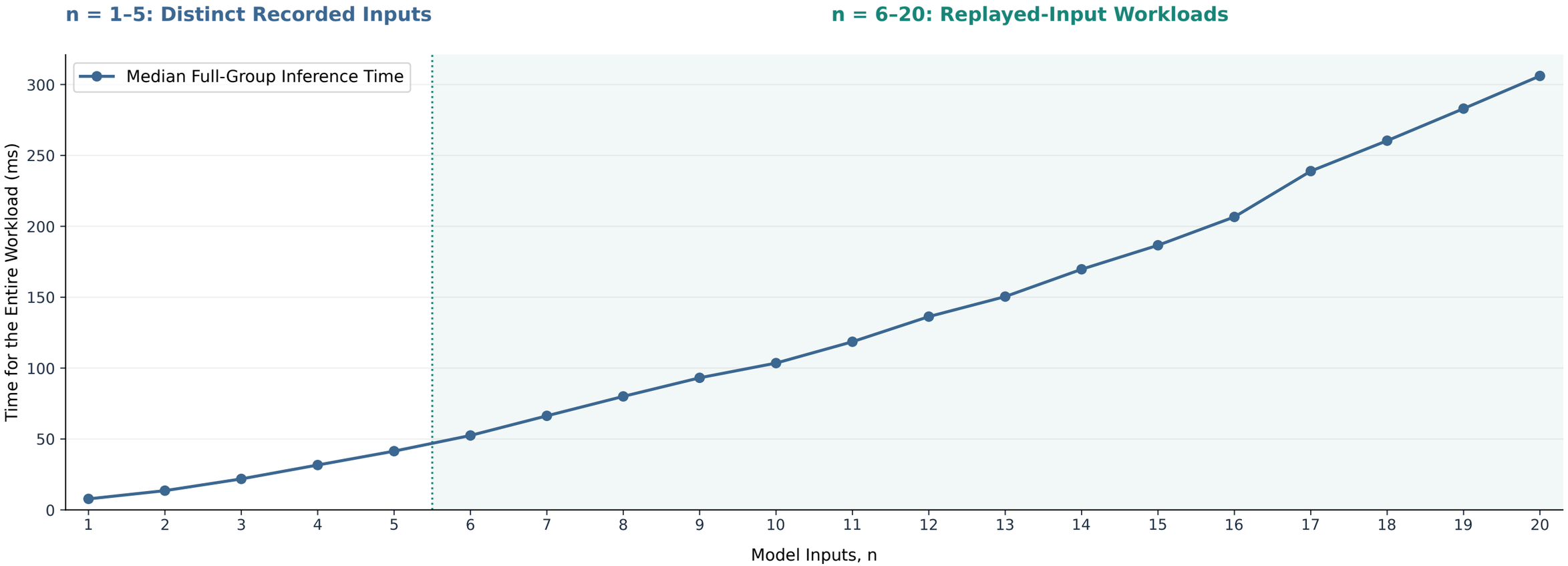
All rows use n active model slots on one GPU. n = 6–20 is a replayed-input runtime workload.

Complete n = 1–20 sweep: 7 min 29 sec after device setup · Runtime only; no AP.

Median forward time includes encoder, collaboration, decoder and heads; input transfer, file reads and CPU packing are excluded.

Custom Experiment 1: Runtime Versus Model Input Count

V2X-Sim 2.0 · 30 shared frames · NVIDIA GeForce RTX 5060 Ti · All n = 1–20 conditions measured



n = 1 → 20: forward median 7.8 → 306.0 ms (39.4×).

Each point is the median complete-forward time across 90 samples, including encoder, fusion, decoder and heads.

Above five inputs, the workload repeats the same five recorded vehicle scans and their poses. No AP is evaluated.

Source: Verified V2X-Sim runtime-workload study · All 1,800 timing samples and source mappings retained in the record

Custom Experiment 2: Coarse Attention and Kruskal Fusion

Compare the original complete graph with a selected tree · Same trained checkpoint · No retraining

Select Cheaply, Then Collaborate at Full Resolution

1. Coarse Attention

Pool each 32 × 32 feature map to 4 × 4.
Align and score every receiver–sender pair.
Normalize over senders at each cell.
Reduce each map to mean squared attention.

2. Maximum Spanning Tree

Average the two directional edge scores.
Kruskal keeps the highest-scoring links
without cycles, until n − 1 remain.
Use both directions of each selected link.

3. Sparse Collaboration

Keep each vehicle's own feature map.
Use original 32 × 32 alignment and scoring
only for its selected tree neighbors.
Renormalize weights, then decode as before.

Full-Resolution Pair Evaluations

Original: n^2 pairs. Tree: n self pairs + 2(n − 1) directed links = 3n − 2.
At n = 15: 225 → 43 fine pairs, plus 225 coarse pairs in one small batch.

Matched Runtime and Accuracy Evaluation

Same 30 frames · Fixed ego 1 · 90 timings per method and n.
Time the full model, including coarse scoring and Kruskal.
AP uses original ego labels, confidence > 0.7 and NMS IoU 0.01.

Only the full-resolution collaboration work becomes linear in n. Coarse pair scoring still scales with n^2.

One fusion round: each vehicle receives its immediate tree neighbors; information does not cross the whole tree in this pass.

n = 1–5 uses distinct recorded V2X-Sim inputs. n = 6–15 repeats the five recorded inputs and poses; AP describes that replay workload.

Source: Verified 30-frame original/tree experiment · Unchanged epoch-100 checkpoint

AI generated illustration

Custom Experiment 2: Runtime and Detection AP

NVIDIA GeForce RTX 5060 Ti · Same 30 ego views · 3 timing trials · Unchanged checkpoint · n = 1–15

Runtime: Median Full-Group Inference Time · AP: Average Precision at Ego 1

n	Original Median (ms)	Tree Median (ms)	Original AP@0.5 (%)	Tree AP@0.5 (%)	Original AP@0.7 (%)	Tree AP@0.7 (%)
1	7.82	7.86	48.91	48.91	45.54	45.54
2	13.34	13.25	60.94	60.94	53.67	53.67
3	21.99	21.41	66.52	60.17	58.71	52.34
4	31.39	29.38	70.80	58.06	63.39	50.37
5	41.57	36.54	74.46	62.65	67.53	53.66
6	52.43	43.63	73.54	57.71	66.61	50.69
7	66.67	52.89	73.93	57.87	66.70	50.57
8	79.83	60.47	73.80	59.16	66.90	52.01
9	92.07	67.48	74.50	61.56	67.06	53.33
10	102.80	70.12	74.46	61.72	67.53	53.66
11	118.76	76.93	74.00	58.57	67.05	50.44
12	133.49	82.84	74.17	58.77	67.20	50.72
13	147.97	86.52	74.36	59.50	67.18	51.61
14	165.79	92.73	74.90	60.93	67.50	52.75
15	185.05	99.25	74.46	60.48	67.53	52.45

At n = 15: 46.4% less inference time; AP changes -13.98 points at IoU 0.5 and -15.08 at IoU 0.7.

Blue rows: distinct recorded inputs (n ≤ 5). Green rows: five recorded inputs replayed into n model slots (n > 5).

Lower runtime and higher AP are better. n = 1 and 2 have identical graphs and predictions; their timing differences are measurement variation.

90 timing samples per entry · 427 reference car boxes at ego 1 · Selection included; input transfer and postprocessing excluded.

Source: Saved matched original/tree timings and detections; every AP and median recomputed before publication